Projekt 1 – orkiestracja zadań za pomocą *Apache Airflow* (docker)

Apache Airflow to platforma *open source* do opracowywania, harmonogramowania i monitorowania wsadowych przepływów zadań (*workflows*). Funkcjonalność, którą ona oferuje, często określa się mianem orkiestracji.

Niektórzy uważają, że jest to jedno z najlepszych narzędzi w swojej klasie.

Apache Airflow jest oparte na Pythonie. Prawie w każdym elemencie systemu wykorzystywany jest właśnie ten język programowania. Począwszy od sposobu definiowania przepływów przez użytkownika, a skończywszy na implementacji wtyczek. Programistów często zaskakuje zakres możliwości *Apache Airflow*, który pozwala na tworzenie przepływów, wykorzystujących praktycznie dowolną technologię.

Istotnym elementem *Apache Airflow* jest interfejs sieciowy. Pozwala on nie tylko podglądać zdefiniowane przepływy, oraz je uruchamiać, ale także w pełnym zakresie zarządzać ich stanem.

Apache Airflow można wdrożyć na wiele sposobów, od pojedynczego procesu na laptopie po konfigurację rozproszoną, która obsługuje potężne przepływy w dużej ich liczbie. My wykorzystamy bardzo prostą konfigurację uruchomioną w ramach kontenera master naszego środowiska.

Celem tego warsztatu będzie przygotowanie przepływu, który będzie zarządzał uruchamianiem (orkiestrował) elementów składowych projektu 1, a następnie przygotowanie pełnego zestawu składowych do rejestracji projektu w ramach kursu.

Elementy wyjściowe

W założeniu wszystkie elementy składowe dla projektu 1 są już gotowe. W przypadku niniejszego warsztatu będzie to wariant: *Hadoop Streaming, Zestaw danych nr 0*.

W założeniu mamy już gotowe:

- Program *MapReduce*:
  - skrypt mappera: `mapper.py`
  - skrypt reduktora: `reducer.py`
  - skrypt uruchamiający program *MapReduce*: `run_mr.sh`
- Program Hive
  - skrypt z poleceniami HQL: `hive.hql`
  - skrypt uruchamiający program Hive: `run_hive.sh`
- Dane wejściowe
  - dostępne w ich oryginalnym miejscu wskazanym w ramach kursu

Ponadto zakładamy, że dostępna jest źródłowa definicja przepływu *Apache Airflow*, plik `projekt1.py`

Zwróć uwagę, że każdy projekt jest inny, oparty o inny zestaw danych, zawierający składowe specyficzne dla określonego wariantu. Dlatego nie traktuj tego warsztatu „dosłownie”. Każdorazowo uwzględnij elementy specyficzne dla Twojego projektu i odpowiednio dostosuj podejmowane działania. W razie potrzeby szukaj wsparcia u prowadzącego. Nie zmieniaj jednak elementów wspólnych. Nie używaj w projekcie elementów które by Ciebie identyfikowały

Wszystkie elementy składowe projektu powinny być wcześniej przetestowane a ich poprawne działanie powinno być potwierdzone. Polecenia uruchamiające poszczególne programy powinny być opracowane, znane i działające.

© 2025 Krzysztof Jankiewicz

W razie potrzeby zanim zaczniesz ten warsztat wprowadź odpowiednie poprawki oraz przetestuj jeszcze raz wszystkie składowe projektu 1. Alternatywnie możesz próbować to robić na bieżąco… choć może to być trudniejsze.

Przygotowanie środowiska

1. Uruchom maszynę, w której znajduje się przygotowane środowisko.

2. Otwórz terminal linii komend

3. Sprawdź i ewentualnie uruchom nasze środowisko

   a. W terminalu przejdź do katalogu `BigData25` zawierającego konfigurację naszego środowiska *Dockerowego*

```
cd ~/BigData25
```

```
admin@debian:~$ cd ~/BigData25
admin@debian:~/BigData25$ ls docker-compose.yml
docker-compose.yml
```

   b. Uruchom polecenie sprawdzające jakie kontenery w ramach naszego środowiska już istnieją i czy zostały one już uruchomione.

```
docker compose ps -a --format "{{.ID}} {{.Image}} {{.Names}} {{.Status}}"
```

   c. Jeśli jakieś kontenery istnieją, to przed dalszymi działaniami (o ile nie kontynuujesz wcześniejszej pracy) usuń nasze środowisko za pomocą poniższego polecenia

```
docker compose down -v
```

   d. Dokonaj edycji naszego pliku konfiguracyjnego dodając do kontenera master jeszcze jeden port, `- "8080:8080" # Apache Airflow` który będzie dostępny z poziomu kontenera master

```
ports:
  - "4040:4040" # Spark Driver UI
  - "8020:8020" # HDFS NameNode RPC port
  - "8080:8080" # Apache Airflow
  - "8088:8088" # YARN ResourceManager UI
  - "9870:9870" # HDFS NameNode Web UI
  - "9999:9999" # Tez UI (static files)
```

```
nano docker-compose.yml
```

   e. Zapisz zmiany, a następnie wyjdź edytora. Uruchom nasze środowisko

```
docker compose up -d
```

4. Upewnij się czy wszystkie kontenery zostały poprawnie uruchomione

```
docker compose ps -a --format "{{.ID}} {{.Image}} {{.Names}} {{.Status}}"
```

```
admin@debian:~/BigData25$ docker compose ps -a --format "{{.ID}} {{.Image}} {{.Names}} {{.Status}}"
e6382e644693 bigdata25-master:latest bigdata25-master-1 Up 14 seconds
f94485712435 mysql:8.4 bigdata25-metastore-1 Up 14 seconds
20c6945e7b9b bigdata25-worker:latest bigdata25-worker1-1 Up 14 seconds
faa8a60b6676 bigdata25-worker:latest bigdata25-worker2-1 Up 14 seconds
```

Jeśli wszystkie 4 kontenery działają poprawnie, możemy przejść dalej.

© 2025 Krzysztof Jankiewicz

Instalacja i uruchomienie Apache Airflow

Zgodnie z dokumentacją, *Apache Airflow* wymaga zainstalowanego Pythona w wersji 3.X. Sama instalacja natomiast powinna się odbywać za pomocą instalatora pakietów Pythona `pip`. Postąpimy zatem zgodnie z tymi regułami.

5. Zalogujemy się teraz do serwera master naszego klastra.

```
docker compose exec -it master bash
```

6. Sprawdź za pomocą poniższego polecenia wersję zainstalowanego Pythona

```
python --version
```

7. Zanim zainstalujemy samego *Apache Airflow*, ustaw kilka zmiennych

```
export AIRFLOW_HOME=~/airflow
export AIRFLOW_VERSION=3.1.0
export AIRFLOW__CORE__LOAD_EXAMPLES=False
export PYTHON_VERSION="$(python -c 'import sys; \
print(f"{sys.version_info.major}.{sys.version_info.minor}")')"
export CONSTRAINT_URL="https://raw.githubusercontent.com/apache/airflow/\
constraints-${AIRFLOW_VERSION}/constraints-${PYTHON_VERSION}.txt"
```

8. Dodaj do zmiennej środowiskowej ścieżkę do narzędzi Apache Airflow

```
export PATH=$PATH:~/.local/bin
```

9. Zainstaluj narzędzie uv

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

10. Zainstaluj wirtualne środowisko za jego pomocą

```
uv venv
```

11. Zainstaluj *Apache Airflow*

```
uv pip install "apache-airflow==${AIRFLOW_VERSION}" --constraint "${CONSTRAINT_URL}"
```

    Nie będziemy instalowali żadnych dodatkowych pakietów dostarczających dodatkowe operatory z niezliczonych możliwości. Wystarczą nam podstawowe.

12. Aktywuj wirtualne środowisko utworzone przez narzędzie uv

```
source ~/.venv/bin/activate
```

13. Zainicjuj bazę danych, z której będzie korzystał *Apache Airflow*

```
airflow db migrate
```

14. Skoryguj wartość parametru decydującego o częstotliwości odświeżania listy przepływów

```
sed -i 's/^refresh_interval\s*=.*/refresh_interval = 15/' \
./airflow/airflow.cfg
```

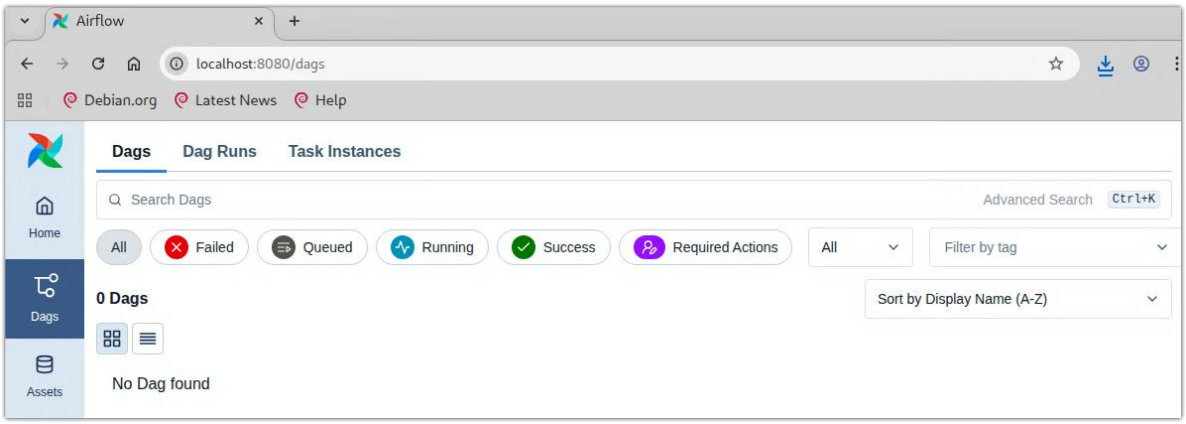
© 2025 Krzysztof Jankiewicz

15. Czas na finał – uruchom naszego *Apache Airflow*. Pozostaw ten terminal otwarty. Zwróć uwagę na dane uwierzytelniające administratora. Warto je sobie przekopiować, zaraz z nich skorzystamy.

```
airflow standalone
```

```
(hadoop) hadoop@master:~$ airflow standalone
standalone | Starting Airflow Standalone
Simple auth manager | Password for user 'admin': C5wFUC3kyz23SdVB
standalone | Checking database is initialized
2025-10-06T13:08:32.462354Z [info     ] Migrating the Airflow database [airflow.utils.db] loc=db.py:1129
standalone | Database ready
api-server | 2025-10-06T13:08:34.213772Z [info     ] Running the uvicorn with:
api-server | Apps: all
api-server | Workers: 1
api-server | Host: 0.0.0.0:8080
api-server | Timeout: 120
```

16. Otwórz stronę webową *Apache Airflow* dostępną pod adresem `localhost:8080`



Kilka przydanych informacji

Może się zdarzyć, że terminal, na którym został uruchomiony Apache Airflow przestanie działać. Jeśli tak się stanie, procesy mogą zajmować porty potrzebne do jego ponownego uruchomienia. Przydatne polecenia poniżej

```
sudo ss -tulnp | grep <PORT>
sudo kill -9
```

W przypadku gdybyśmy zapomnieli naszego hasło, zostało ono zapisane w pliku i możemy je odzyskać za pomocą polecenia

```
head ~/airflow/simple_auth_manager_passwords.json.generated
```

Instalacja przepływu

17. Pobierz na maszynę, z której korzystasz, do bieżącego katalogu, komplet składowych swojego projektu. Jeśli nie masz pomysłu jak to zrobić, skorzystaj z pomocy prowadzącego.

18. Korzystając z nowego terminala linii komend, przejdź do katalogu, w którym zapisane zostały te pliki i upewnij się, że wszystkie one są dostępne.

```
admin@debian:~/Pobrane$ ls
hive.hql  mapper.py  reducer.py  run_hive.sh  run_mr.sh
```

© 2025 Krzysztof Jankiewicz

19. Przekopiuj te pliki do naszego kontenera master

```
docker cp ./ bigdata25-master-1:/home/hadoop/
```

20. Przełącz się na terminal z aktywną sesją w kontenerze master i sprawdź dostępność naszych plików

```
hadoop@master:~$ ls
airflow  hive.hql  mapper.py  notebooks  reducer.py  run_hive.sh  run_mr.sh
```

21. Utwórz w katalogu `~/airflow` podkatalog `dags`, z którego będą załadowywane definicje przepływów. Przy okazji, utwórz w nim podkatalog `project_files`, który będzie zawierał wszystkie składowe naszego projektu

```
mkdir -p ~/airflow/dags/project_files
```

22. Pobierz źródłową wersję przepływu

```
curl -u student https://jankiewicz.pl/studenci/bigdata/projekt1/projekt1.py -O
```

23. Otwórz w wygodnym dla Ciebie edytorze definicję tego przepływu.

24. Na początku zwróć uwagę na parametry, które ten przepływ wykorzystuje. Dokonaj korekty domyślnej wartości parametru `classic_or_streaming` na zgodną z Twoim wariantem projektu. W tym przypadku, dla warsztatowego zestawu (*Hadoop Streaming, Zestaw danych nr 0*), pozostawiamy wartość `streaming`.

```
11    params={
12        "dags_home": Param(
13            "TU UZUPELNIJ SCIEZKE DLA TWOJEGO KATALOGU AIRFLOW/airflow/dags", type="string"
14        ),
15        "input_dir": Param(
16            "TU UZUPELNIJ SCIEZKE DLA TWOICH DANYCH ZRODLOWYCH/input", type="string"
17        ),
18        "output_mr_dir": Param("/project1/output_mr3", type="string"),
19        "output_dir": Param("/project1/output6", type="string"),
20        "classic_or_streaming": Param(
21            "streaming", enum=["classic", "streaming"]
22        ),
23    },
```

Nie zmieniaj wartości pozostałych parametrów. Będziemy je korygowali podczas uruchamiania przepływu. Znaczenie tych parametrów jest następujące:

a. `dags_home` – katalog `dags`, w którym znajduje się podkatalog `project_files` ze wszystkimi plikami źródłowymi naszego projektu
b. `input_dir` – katalog `input` zawierający oba zbiory danych naszego zestawu `datasource1` oraz `datasource4`
c. `output_mr_dir` – katalog HDFS, wynikowy dla zadania *MapReduce*
d. `output_dir` – katalog HDFS, wynikowy dla *Hive* (końcowy wynik naszego przetwarzania)

© 2025 Krzysztof Jankiewicz

25. Operatory `clean_output_mr_dir` oraz `clean_output_dir` mają na celu usunięcie wyników poprzednich uruchomień naszego przepływu i nie powinny być zmieniane.

```
    # Usuwanie katalogów z HDFS jeśli istnieją
    clean_output_mr_dir = BashOperator(
        task_id="clean_output_mr_dir",
        bash_command=(
            "if hadoop fs -test -d {{ params.output_mr_dir }}; "
            "then hadoop fs -rm -f -r {{ params.output_mr_dir }}; fi"
        ),
    )

    clean_output_dir = BashOperator(
        task_id="clean_output_dir",
        bash_command=(
            "if hadoop fs -test -d {{ params.output_dir }}; "
            "then hadoop fs -rm -f -r {{ params.output_dir }}; fi"
        ),
    )
```

26. Operator `pick_classic_or_streaming` powinien wybrać kolejny operator do wykonania na podstawie wartości parametru `classic_or_streaming`. Jego także nie zmieniamy

```
    # Wybór trybu wykonania: klasyczny MR lub streaming
    def _pick_classic_or_streaming(params):
        if params["classic_or_streaming"] == "classic":
            return "mapreduce_classic"
        else:
            return "hadoop_streaming"

    pick_classic_or_streaming = BranchPythonOperator(
        task_id="pick_classic_or_streaming",
        python_callable=_pick_classic_or_streaming,
        op_kwargs={"params": dag.params},
    )
```

27. Teraz dość istotny moment. W zależności od tego jaki wariant programu *MapReduce* jest przez Ciebie realizowany, dostosuj odpowiednio wywołanie zadania *MapReduce* albo w operatorze `mapreduce_classic` albo `hadoop_streaming`. Oprzyj się na wywołaniu Twojego programu używanego do tej pory (powinno ono być zapisane w pliku `run_mr.sh`). Pamiętaj o parametryzacji, korzystaj z dostępnych parametrów przepływu.

```
    # MapReduce klasyczny
    mapreduce_classic = BashOperator(
        task_id="mapreduce_classic",
        bash_command=(
            "hadoop jar {{ params.dags_home }}/project_files/ . . . "
        ),
    )

    # MapReduce streaming
    hadoop_streaming = BashOperator(
        task_id="hadoop_streaming",
        bash_command=(
            "mapred streaming "
            "-files {{ params.dags_home }}/project_files/ . . ."
        ),
    )
```

W przypadku warsztatowego zestawu (*Hadoop Streaming, Zestaw danych nr 0*) będzie korygowany operator `hadoop_streaming`.

© 2025 Krzysztof Jankiewicz

28. Ponownie istotny moment. Uzupełnij w operatorze `hive` wywołanie programu *Hive*. Pamiętaj o parametryzacji, korzystaj z dostępnych parametrów przepływu. Oprzyj się na wywołaniu używanym do tej pory (powinno ono być zapisane w pliku `run_hive.sh`).

```
75    # Program Hive
76    hive = BashOperator(
77        task_id="hive",
78        bash_command=(
79            "beeline -u jdbc:hive2://localhost:10000/default "
80            ". . ."
81        ),
82        trigger_rule="none_failed",
83    )
```

29. Ostatni operator `get_output` pobiera wyniki naszego je przetwarzania i je prezentuje. Jego także nie zmieniamy.

```
85    # Pobranie wyników
86    get_output = BashOperator(
87        task_id="get_output",
88        bash_command=(
89            "hadoop fs -getmerge {{ params.output_dir }} output6.json && cat output6.json"
90        ),
91        trigger_rule="none_failed",
92    )
```

30. Przyglądnij się temu jak wyglądają wzajemne zależności pomiędzy tymi operatorami.

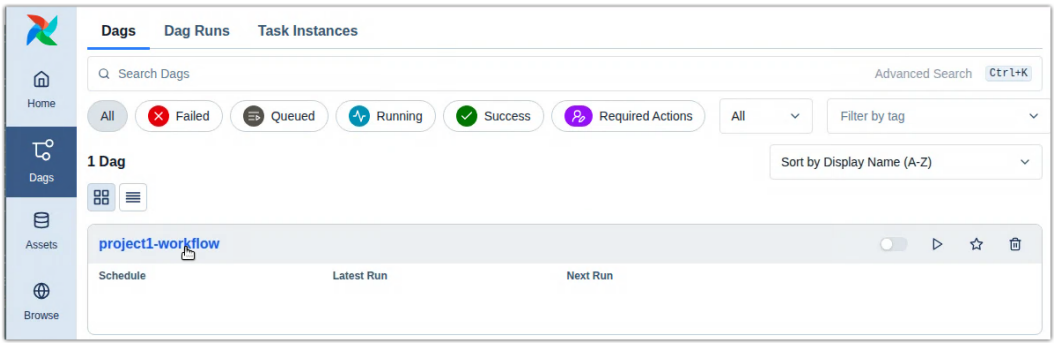
```
94    # Zależności
95    [clean_output_mr_dir, clean_output_dir] >> pick_classic_or_streaming
96    pick_classic_or_streaming >> [mapreduce_classic, hadoop_streaming]
97    [mapreduce_classic, hadoop_streaming] >> hive
98    hive >> get_output
```

31. Jeśli uważasz, że musisz coś koniecznie dodać w tym przepływie, masz do tego pełne prawo. Zastanów się jednak dokładnie czy te działania są niezbędne, czy nie wprowadzą niepotrzebnych opóźnień. Wykonywanie zbędnych operacji, wykorzystujących niepotrzebnie zasoby, jest błędne.

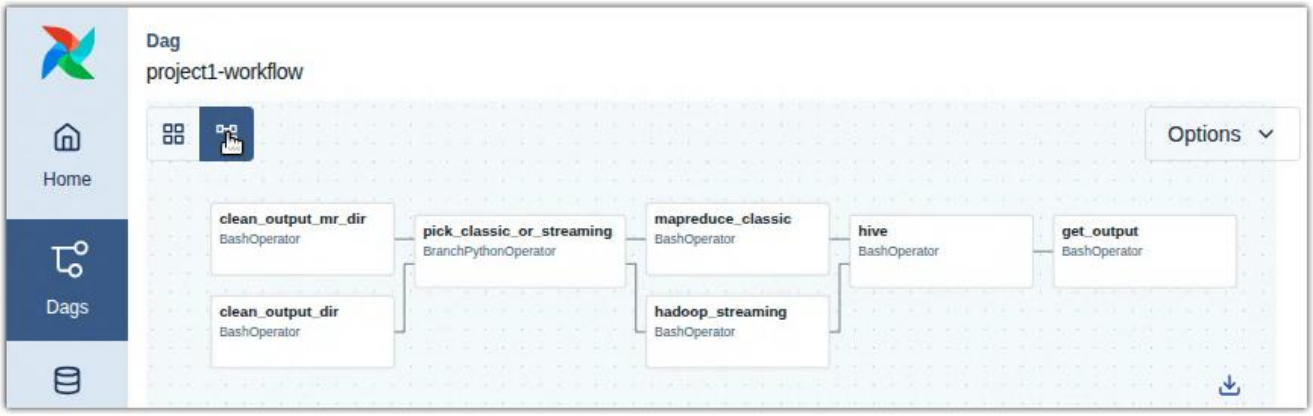
32. Przenieś plik z definicją przepływu do katalogu `dags`. Pozostałe pliki projektu przenieś do katalogu `project_files`

```
mv projekt1.py ~/airflow/dags/
mv *.* ~/airflow/dags/project_files
```

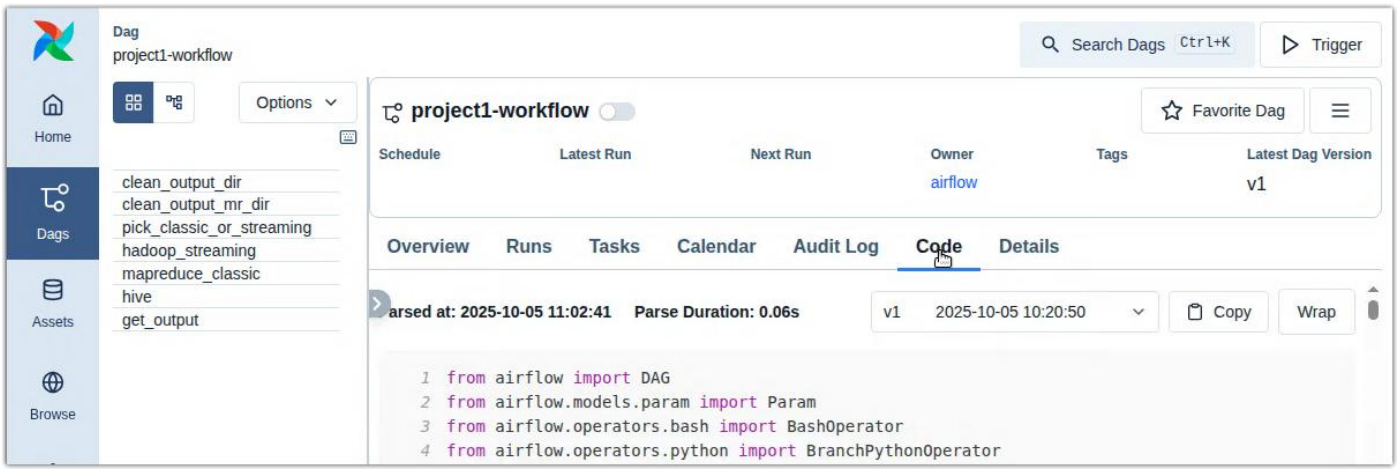
33. Przejdź do interfejsu webowego *Apache Airflow*. Znajdź przepływ przeznaczony dla naszego projektu. *Apache Airflow* przegląda co jakiś czas katalog z definicjami przepływów. Może upłynąć kilka minut zanim nowa definicja zostanie zaczytana.



34. Wybierz go, a następnie przyglądnij się jego graficznej reprezentacji



35. Zaglądnij także do kodu tego przepływu i sprawdź czy wszystko się w nim zgadza.



36. Jeszcze raz zwróć uwagę na:
    a. to, że jest on oparty na dwóch typach operatorów
        i. `BashOperator` – który uruchamia polecenia systemu operacyjnego, oraz
        ii. `BranchPythonOperator` – stosowany do warunkowego uruchamiania kolejnych operatorów
    b. parametry oraz ich wartości domyślne,
    c. polecenia zawarte w operatorach `BashOperator` oraz wykorzystanie w nich wartości parametrów.

37. Jeśli zauważysz jakieś niedociągnięcia możesz ponownie dokonać edycji tego przepływu za pomocą wybranego edytora.

38. Powróć do interfejsu sieciowego *Apache Airflow*. Odśwież kod przepływu, sprawdź czy wszystkie wprowadzone zmiany są widoczne. Jeśli Twój przepływ jest zgodny z oczekiwaniami, czas na jego uruchomienie.

© 2025 Krzysztof Jankiewicz

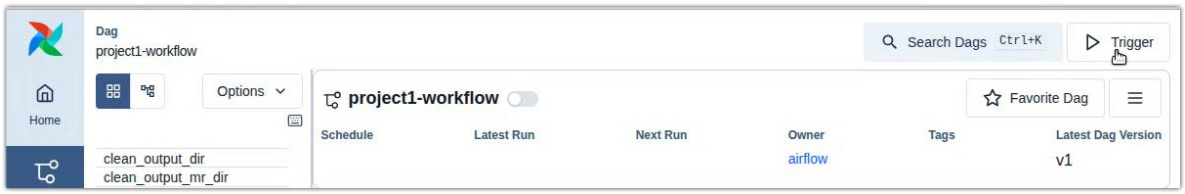
Uruchomienie przepływu

39. Aby móc uruchomić nasz przepływ potrzebujemy jeszcze jednego składnika – danych wejściowych. Pobierz je, a następnie rozpakuj i umieść w wybranym przez Ciebie katalogu w HDFS. Ponownie zwróć uwagę, że warsztatowym zestawem jest *Hadoop Streaming, Zestaw danych nr 0*. Twój z pewnością jest inny.

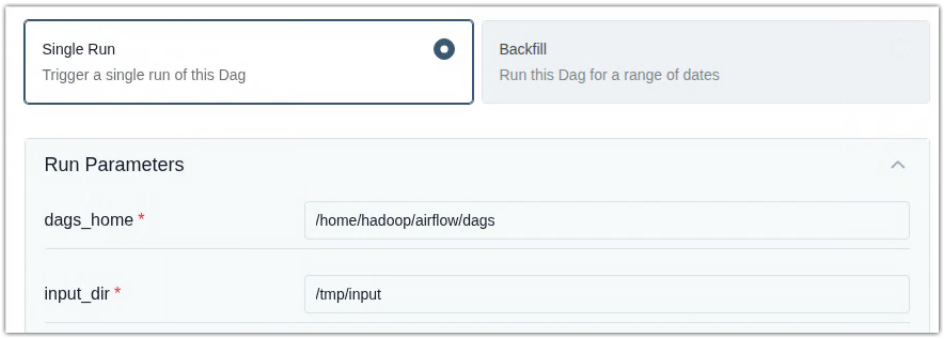
```
unzip zestaw0.zip
hadoop fs -put input /tmp/
```

Zapamiętaj w jakim katalogu został umieszczony Twój katalog `input` zawierający oba zbiory źródłowych danych datasource1 i datasource4. W powyższym przypadku będzie to `/tmp/input`.

40. Powróć do interfejsu sieciowego *Apache Airflow*, a następnie uruchom nasz przepływ za pomocą przycisku *Trigger*



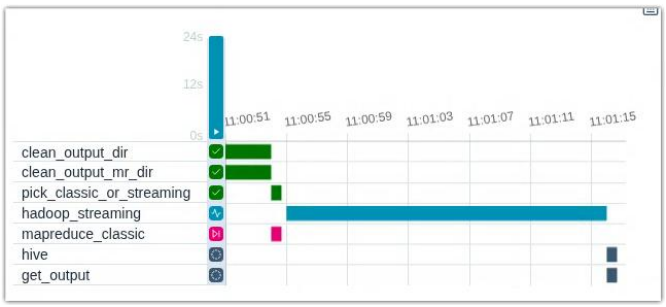
41. Nasze domyślne ścieżki zawarte w parametrach dags_home oraz input_dir nie odpowiadają rzeczywistości, dlatego koniecznie musimy je poprawić. W naszym przypadku będzie to wyglądało następująco.



42. Uruchom przepływ za pomocą przycisku *Trigger*.

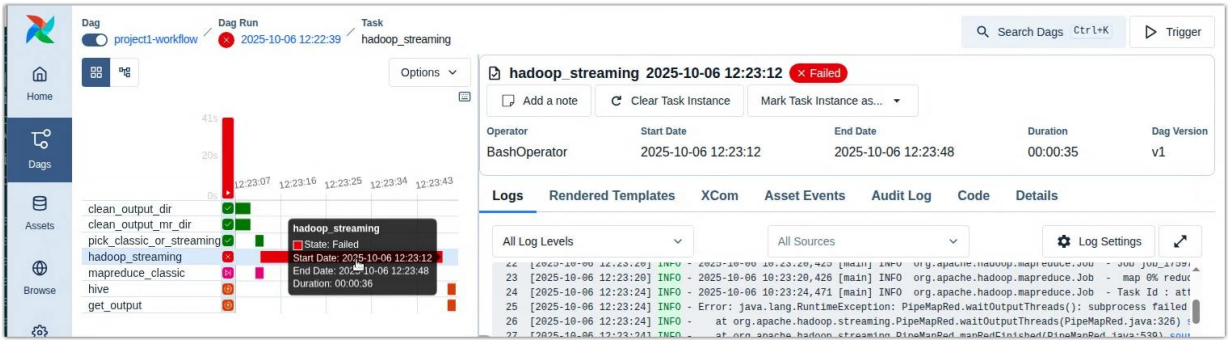


43. Poczekaj aż Twój przepływ zakończy swoje przetwarzanie.



© 2025 Krzysztof Jankiewicz

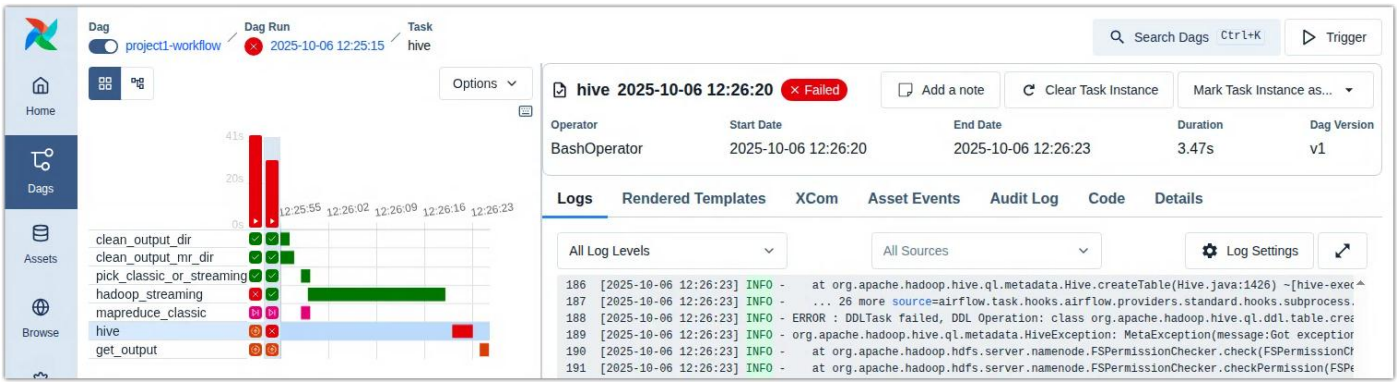
44. Podglądaj przebieg przepływu na zakładce *Grid*. Jeśli Twój przepływ za pierwszym zakończy się sukcesem… no cóż, przyjmij gratulacje. To jednak za pierwszym razem zazwyczaj się nie zdarza. Jeśli pojawi się błąd w ramach któregoś z tasków (uruchomionych w ramach określonego operatora) zaglądnij do logów, aby dowiedzieć się jaka była przyczyna tego błędu.



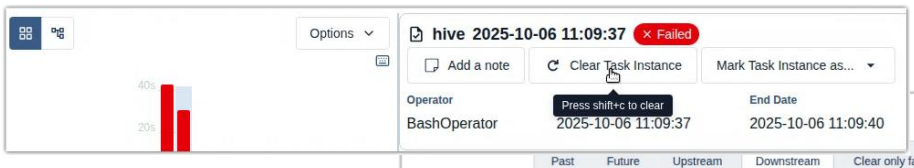
Przykładowo powyżej błąd wygenerowały mappery w ramach *Hadoop Streaming*. Analiza logów z YARN wykazała klasyczny problem znaków nowej linii. Ktoś najwyraźniej nie testował dokładnie swojego projektu.



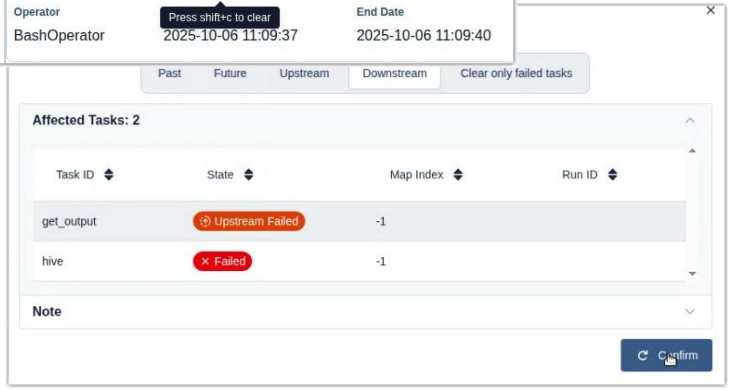
45. Po każdorazowej naprawie błędu możesz ponownie uruchomić nową instancję przepływu, lub wznowić wykonywanie bieżącej instancji korygując status wykonywanej uprzednio jednostki. Poniżej uruchomiono nową instancję przepływu. Niestety dał znać kolejny błąd na poziomie *tasku* operatora `hive`.



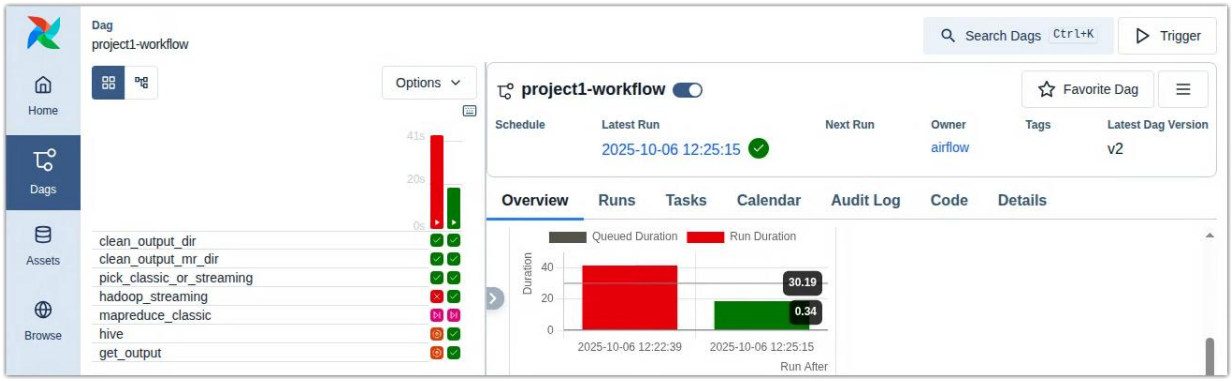
46. Po wyeliminowaniu przyczyny błędu wyczyścimy instancję taska, a wraz z nią wszystkich tasków „w dole” strumienia naszego przepływu.



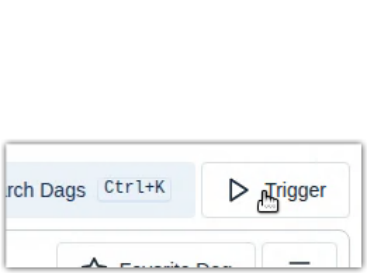
Spowoduje to wznowienie obsługi tej samej instancji przepływu.

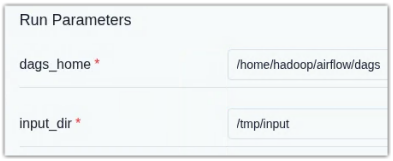


47. W rezultacie powinieneś doprowadzić do tego, że twój przepływ zostanie wykonany w całości.



48. Aby być pewnym powtarzalności uzyskanego efektu uruchom kompletny przepływ jeszcze raz.
Brak powtarzalności może świadczyć o braku pewnych poleceń (np. usuwających wcześniej utworzone konstrukcje) lub użyciu poleceń, które są destrukcyjne i uniemożliwiają ponowne wykonanie przepływu, np. w wyniku usuwania danych źródłowych.



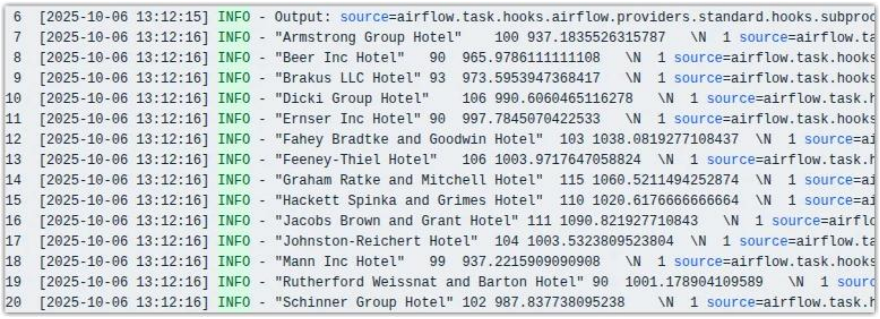




49. Konieczne zaglądnij do logów ostatniego z tasków (`get_ouput`). Znajdziesz w nim zaprezentowany końcowy wynik Twojego przetwarzania. Jeśli go tam nie znajdujesz, to oznacza, że mimo wszystko coś z Twoim przetwarzaniem nie jest tak. Znajdź przyczynę błędu i ją wyeliminuj. Pamiętaj o efekcie powtarzalności.

Jeśli wszystko działa jak należy, mamy gotową całość. Wystarczy teraz ją przygotować i zarejestrować jako rozwiązanie projektu.





Przygotowanie projektu do rejestracji

50. Jeśli Twój projekt jest gotowy, należy przygotować go do rejestracji. W terminalu na maszynie master zainstaluj program `zip`

```
sudo apt update
sudo apt install zip
```

51. Zatrzymaj procesy Apache Airflow

```
pkill -f "airflow"
```

52. Możesz się upewnić czy któryś z tych procesów jeszcze działa

```
hadoop@master:~/airflow$ ps
    PID TTY          TIME CMD
    159 pts/0    00:00:00 bash
   9346 pts/0    00:00:00 ps
[1]+  Terminated              airflow standalone > ~/airflow/standalone.log 2>&1  (wd: ~)
(wd now: ~/airflow)
```

53. Następnie przejdź do katalogu `~/airflow`, a następnie spakuj katalog `dags` wraz z jego zawartością do pliku `projekt1.zip`

```
cd ~/airflow
rm -r dags/__pycache__
zip -r ~/projekt1.zip dags
```

54. Koniecznie upewnij się, że w pliku znalazły się wszystkie elementy wymagane przez projekt, nawet jeśli nie są wykorzystywane w ramach przepływu (przykładowo `run_mr.sh` oraz `run_hime.sh`, albo definicje klas w przypadku wariantu *MapReduce Classic*).

55. Tak przygotowany plik `projekt1.zip` skopiuj w bezpieczne i trwałe miejsce. Możesz go także od razu zarejestrować w ramach swojego projektu na platformie kursu.

    Mając tak przygotowany plik projektu, możemy być pewni, że osoba, która będzie go oceniała, wystarczy, że rozpakuje go do katalogu `~/airflow` i już będzie mogła testować nasz projekt.

© 2025 Krzysztof Jankiewicz